Week 4 Report — Monitoring, Drift Detection & Retraining

Arnav Mahale · AIPI 561 Operationalizing AI · Summer 2026

Baseline window: **Jan 1-15, 2026** (82,080 rows). Candidate window: **Feb 2-28, 2026** (147,744 rows). Both windows come from `demand_enriched_week4.parquet`. Code lives in `week4/scripts/` and runs hourly via `.github/workflows/monitor-drift.yml`.

1. Drift patterns detected

#	Pattern	Statistical evidence	Type	Operational impact
1	**Lag/rolling feature collapse** — `roll_mean_1day` mean 13.82 → 8.15 (-41%); `lag_1day` -40.6%; `lag_1week` -30.0%	KS=0.224, p≈0, PSI=0.352 (`roll_mean_1day`); 52 of 57 zones dropped >30%	Feature drift → concept drift	The model was trained on lag values centered around 14; it now sees values centered around 8. Predictions are systematically biased.
2	**Mid-morning demand collapse** — hours 9-10 dropped 41-43%; hours 1-4 dropped 23-33%	Per-hour mean trip_count comparison; 7 of 24 hours shifted >20%	Segment drift (hour)	Predictions for 9-10am are now too high by ~45%. Affects fleet positioning during peak commute.
3	**Weekend / Monday crash** — Mon -29%, Sat -29%, Sun -38%; Tue-Fri stable	Per-dow mean comparison; 3 of 7 days shifted >20%	Segment drift (dow)	The weekday-vs-weekend signal the model learned no longer holds — calibration broken at the day-of-week level.
4	**Per-zone concentrated drops** — 6 zones shifted >20%, including JFK (zone 132, -17%), zone 230 (-22%), zone 195 (-48%)	Per-zone mean comparison + KS	Segment drift (zone)	Specific zones (airports, peripheral) need either retraining or per-zone overrides.
5	**Overall `trip_count` distribution shift** — mean 14.07 → 12.56 (-10.8%)	KS=0.0425, $p=3.4\times10^{-83}$	Data drift (target)	Bottom-line signal: less demand overall. By itself not catastrophic, but combined with #1-4 it's clear the world changed.
6	**Holiday rate halved** —	KS=0.0296, $p=10^{-40}$	Data drift (categorical rate)	Explains part of #5 (Feb has fewer

#	Pattern	Statistical evidence	Type	Operational impact
	`is_holiday` 6.7% → 3.7%			holidays per row), but rate change still affects holiday-mode predictions.

Important nuance — KS and PSI sometimes disagree. Pattern #5 has KS p ≈ 0 (overwhelmingly significant) but PSI = 0.014 (well below the 0.25 threshold). PSI uses 10 equal-frequency bins, which smears small-but-consistent shifts; KS picks up any divergence in the CDF. The monitoring code reports both; alerts fire on either. See `metrics.metric_4_ks_test` + `metric_5_psi`.

2. Monitoring framework — 8 metrics

Implementation: `week4/scripts/metrics.py`. Eight metrics are exposed (README required ≥ 5); all run inside `MetricComputer.compute_all`. Thresholds come from `BASELINE_METRICS.md`.

#	Metric	What it computes	Segmentation	Threshold (warn / crit)
1	`accuracy_proxy`	% predictions within ±50% of actual (or ±2 trips when small), using baseline (zone, hour, dow) mean as prediction stand-in	global + per-zone	crit when worst zone < 80%
2	`accuracy_by_zone`	Same calc, reported per-zone (count of zones below threshold)	per zone (57)	crit when any zone < 80%
3	`null_rates`	Null fraction per critical column + per lag column	per column	warn > 0.5%, crit > 1% (or 2% on lag cols)
4	`ks_test`	scipy `ks_2samp` on `trip_count`, `roll_mean_1day`, `lag_1day`	per feature	warn p < 0.05, crit p < 0.01
5	`psi`	PSI with 10 quantile bins on the same 3 features	per feature	warn > 0.10, crit > 0.25
6	`mean_shift`	Relative shift of `trip_count` mean vs baseline	global	warn > ±10%, crit > ±20%
7	`duplicate_rate`	Fraction of rows duplicating a `(zone, time_bucket)` key	global	warn > 0, crit > 0.5%
8	`data_freshness`	Age (hours) of latest record vs reference	global	warn > 2h, crit > 24h

Each metric returns a `MetricResult` with `name`, `value`, `breach`, and `detail`. `compute_all` aggregates these and reports overall breach severity. **Segmentation is non-optional** for metrics 2 and 4-5 because, per the reading, "global metrics hide failures."

3. Monitoring schedule — daily at 09:00 UTC

`cron: '0 9 * * *'` plus push-trigger on script changes plus `workflow_dispatch`.

Why daily, not hourly: the most informative metric (`accuracy_proxy`) requires ground-truth pickups. In real upstream pipelines, ground truth lags predictions by 24-48 hours. Running hourly would re-validate the same labelled window and burn ~24× the CI budget for zero new signal. Inversely, weekly is too slow — a degraded model serving for 6 days at 75% accuracy is unacceptable.

Why 09:00 UTC specifically: it lands in EU mid-morning and just before the US East Coast workday, so alerts hit Slack/issues when operators can actually read them, not at 3am.

The validator also runs on push when `week4/scripts/**` changes — catches code regressions before they ship — and is exposed via `workflow_dispatch` for manual ad-hoc runs.

4. Retraining strategy

Triggers

Retrain when **any** of these fires:

Trigger	Threshold
`accuracy_proxy` overall drops below baseline	< 80% (vs. baseline ~91%)
KS p-value on `trip_count` or `roll_mean_1day`	< 0.01
PSI on `trip_count` or `roll_mean_1day`	> 0.25
≥ 5 zones with per-zone accuracy < 80%	persistent for ≥ 2 consecutive daily runs
Schedule fallback	every 14 days regardless

A **single** day with one breach opens a `drift-alert` issue but does **not** auto-retrain — false positives are common. Two consecutive days of breaches escalates to retrain.

Pipeline

```
detect (CI workflow alert) → freeze current model (tag v_n) →
  pull last 30 days of labelled data →
  train candidate v_n+1 →
  offline eval: per-zone accuracy ≥ v_n on a held-out 20% slice →
  shadow deploy v_n+1 for 24h (log predictions, don't serve) →
  canary 10% traffic for 24h →
  compare canary vs production: if v_n+1 accuracy ≥ v_n, promote to 100% →
  retain v_n for 14 days for rollback
```

Each stage has an explicit go/no-go gate; the pipeline halts if a gate fails and surfaces a follow-up issue for human review.

Data window

Train on **last 30 days** (enough to capture weekday × hour × dow profile) plus **the entire most-recent 7 days** weighted 2× (recency bias). 30 days is the smallest window that gives stable per-zone-hour estimates given 57 zones × 96 slots × 7 days; longer windows risk anchoring to the *old* world the drift moved us away from.

Validation

- **Offline:** 80/20 chronological split, last 7 days held out. Reject if per-zone accuracy on held-out is worse than the previous model's on the same slice.
- **Shadow mode:** new model produces predictions in parallel with the live model for 24h, logged but not served. Compare prediction distributions and per-zone accuracy.
- **Canary:** route 10% of traffic to the new model. Promote to 100% only if canary accuracy ≥ live accuracy across all zones over 24h.

Rollback

`kubectl rollout undo deployment/demand-api` reverts the deployment to the previous image (image SHAs are tagged in Artifact Registry per week 2). The previous model is retained for 14 days. Automated rollback fires if the canary drops more than 5 percentage points below the live model on any zone within the first hour.

Versioning

- Images: `us-central1-docker.pkg.dev/.../demand-api:<git-sha>` — already done in week 2's CD.
- Model artifacts: `gs://ops-ai-arnavmahale-data/models/v_<YYYYMMDD>_<sha>.txt`. Metadata sidecar (`.json`) records training window, baseline accuracy per zone, and the drift findings that triggered retraining.
- Retention: keep last 3 models + every retrained version for 90 days.

5. What this design intentionally doesn't do

- **No live model.** `accuracy_proxy` uses the baseline (zone, hour, dow) mean as a prediction; production would call the LightGBM model. Building the real proxy is the natural next step.
- **No correlation drift check.** A feature can stay structurally valid but stop predicting (Pattern #1 hints at this for lag features). Adding a Pearson-r vs baseline check on each feature against the target is the next iteration.
- **No alerting beyond a GitHub issue.** Real prod would route the JSON output to PagerDuty / Slack with severity-based routing rules. The monitor's `metrics-latest.json` artifact is ready for that.
- **Schedule is fixed (daily).** A more sophisticated trigger would run more often during high-volatility windows (post-deployment, major holidays) and less often during stable periods.